

# Green Heterogeneous Computing Powers Allocation Using Reinforcement Learning in SDN-IoV

Yingjie Liu<sup>1b</sup>, Dan Wang<sup>1b</sup>, Bin Song<sup>1b</sup>, *Senior Member, IEEE*, and Xiaojiang Du<sup>1b</sup>, *Fellow, IEEE*

**Abstract**—In the era of 6G-driven Internet of Vehicles (IoV), the form of services centered on computing power has promoted the development of applications such as In-Vehicle Infotainment (IVI) and smart driving assistance. The collaboration of computing power deployed between edge and end provides computing services for IoV applications. However, due to the diversity of computing tasks and the heterogeneity of computing power resources, it is a challenge to allocate computing power resources reasonably to meet user needs. To solve this problem, we propose a green heterogeneous computing power allocation method to reduce the energy consumption. First, we design a Heterogeneous Computing Power-IoV (HCP-IoV) architecture based on Software Defined Network (SDN) to realize the deep integration of computing power and network through the hybrid network architecture of vehicle-road collaboration and road-road collaboration. Then, to ensure the reasonable allocation of different computing tasks, we design an intelligent heterogeneous computing power allocation scheme to reduce energy consumption on HCP-IoV. Furthermore, we formulate the joint communication and computing power allocation as a Markov Decision Process (MDP) and use the improved Proximal Policy Optimization (PPO) method to achieve an adaptive allocation of heterogeneous computing powers. Simulation results show the effectiveness of the proposed method.

**Index Terms**—IoV, heterogeneous computing power, reinforcement learning, multivariate Gaussian distribution.

## I. INTRODUCTION

NOWADAYS, the intelligent demands for services have promoted the development of hardware and software on the Internet of Vehicles (IoV) [1], which enables more delay-sensitive intelligent services to be deployed on vehicles and Road Side Units (RSU) [2], including license plate recognition, Augmented Reality (AR) [3], [4] navigation, vehicles recognition, etc. For example, the XPILOT 3.0 [5] autonomous driving assistance system produced by XPeng Motors used

NVIDIA Xavier to provide the computing power required for vehicles and traffic lights recognition in 2020. They put the trained model into the system to perform the recognition tasks, which can satisfy the real-time requirements of the services. However, the computing power of a single-vehicle cannot meet the delay requirements of 3D scene reconstruction and AR navigation that requires large computing power services. Therefore, it is necessary to consider jointing the computing power on the edge and vehicles to support the above services [6] collaboratively. As an emerging technology, Mobile Edge Computing (MEC) [7], [8] enables to provide computing power and resource scheduling for intelligent services through edge servers deployed in RSUs. It can provide low-latency services for IoV users through efficient caching and computing at edge nodes [9].

Recently, edge intelligence [10], [11] technology combines edge computing and Artificial Intelligence (AI) [12] to realize intelligent services and resource orchestration in IoV. To provide services that can be deployed quickly and efficiently while meeting the requirements of users, a variety of heterogeneous computing powers have emerged to support it [13]. There are several typical chips that provide computing powers, which are CPU, GPU and NPU [14], [15]. Each computing power is designed to support different application services. CPU is mainly excellent in complex logic operations and control, GPU focuses on image processing and parallel computing, and NPU is used for neural network acceleration and tensor calculation. The heterogeneous computing powers are deployed on edge servers and vehicles to compute tasks. Vehicles consider offloading computing tasks to edge servers when they face services with massive computing power demand. Meantime, if the computing powers in edge servers do not meet the demands of users, the tasks should be offloaded to the cloud [16]. However, it is not suitable for delay-sensitive services. Offloading tasks to nearby edge servers with sufficient computing power is a good way to reduce transmission delay and improve resource utilization, but it complicates resource allocation. Meantime, the scheduling of computing powers between different chips also increases the difficulty of resource allocation in MEC [17]. Therefore, a new challenge is brought to network operators: How to rationally schedule resources based on heterogeneous computing power?

Software-defined Network (SDN) [18] architecture can coordinate heterogeneous computing power resources. It is a programmable network architecture divided into a control plane and a data plane. SDN controls global network

Manuscript received 10 December 2021; revised 2 May 2022; accepted 20 June 2022. Date of publication 29 June 2022; date of current version 19 May 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62071354; in part by the Open Research Projects of Zhejiang Lab under Grant 2019KD0AD01/013; in part by the Key Research and Development Program of Shaanxi under Grant 2022ZDLGY05-08; and in part by the ISN State Key Laboratory. (*Corresponding author: Bin Song.*)

Yingjie Liu, Dan Wang, and Bin Song are with the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an 710071, China (e-mail: yjliu\_2@stu.xidian.edu.cn; danwang@xidian.edu.cn; bsong@mail.xidian.edu.cn).

Xiaojiang Du is with the Department of Electrical and Computer Engineering, Stevens Institute of Technology, Hoboken, NJ 07030 USA (e-mail: dxj@ieee.org).

Digital Object Identifier 10.1109/TGCN.2022.3187077

information on the control plane and realizes data forwarding on the data plane. The combination of SDN and edge computing can provide resource management for edge computing nodes, thereby improving resource utilization. Moreover, it increases the flexibility of edge resource scheduling and enhances the management of edge heterogeneous computing power resources.

Energy consumption is a critical problem in 6G-driven IoV, which is related to the profits of network operators and users. In [19], three new services are proposed in 6G, including universal mobile ultra-broadband (uMUB), ultrahigh data density (uHDD), and ultrahigh speed low-latency communications (uHSLLC). The frequency band of 6G is higher than 5G, and the communication rates in 6G IoV will be increased to Tera bps, which has a high demand for energy. Many car manufacturers in the world are developing new energy vehicles to reduce the use of non-renewable energy. Besides, reasonable allocation of computing powers can also reduce the energy consumption of the IoV system. But due to the diversified requirements of delay and energy, developing green computing and communication solutions is a challenge for the realization of 'green IoV' [20].

The 6G IoV requires intelligent management of resources such as communication, computing and storage based on artificial intelligence technology. Although there are various research works on resource allocation in IoV, there are few methods for heterogeneous computing powers. The current solutions for resource allocation mainly include convex optimization and non-convex optimization. (Lagrangian duality method, KKT condition, etc.) [17] Although these methods enable to search for an approximate optimal solution, they cannot adapt to dynamically changing scenarios. Solutions based on game theory [21] are also widely used in resource allocation problems, but Nash equilibrium is not optimal. Due to the recent development of AI technology, Reinforcement Learning (RL) [22]–[24] methods have gradually been applied to various fields. It is suitable for distributed and centralized high-complex resource allocation scenarios. Especially the resource allocation method based on deep RL has a high accuracy rate [25].

In this paper, we investigate the green heterogeneous computing power allocation method in 6G IoV. First, we proposed an HCP-IoV architecture based on SDN. On this basis, we further design a heterogeneous computing power allocation scheme to meet the latency and energy consumption requirements of different types of computing tasks. Finally, the dynamic computing power allocation strategy between vehicles and RSUs is explored through an improved Proximal Policy Optimization (PPO) [26] method. The specific contributions are as follows:

- We propose an HCP-IoV framework based on SDN to realize the reasonable allocation of heterogeneous computing power through vehicle-road coordination and road-road coordination. The proposed framework supports the adaptive allocation of different types of computing powers and meets the needs of vehicle intelligent application services.

- We design a green heterogeneous computing powers allocation scheme under the network architecture of vehicle-road coordination and road-road coordination to realize the deep integration of computing power and network. Also, we formulate an optimization problem of joint communication and computing power to minimize transmission and computing energy consumption while guaranteeing the latency requirements of computing tasks.
- The PPO method is employed to solve the proposed joint optimization problem. We build a Markov Decision Process (MDP) for the interaction of the SDN controller with the computing power network. To improve the applicability of the PPO algorithm to the proposed scheme, we improve the existing PPO algorithm where the optimal decision of each step is output by sampling in the constructed multivariate Gaussian probability density function of heterogeneous computing power allocation. Simulation results show the effectiveness of the proposed method.

## II. RELATED WORK

The IoV can increase the information transmission rate and reduce energy consumption by exchanging information with the servers deployed in RSUs [27]. At present, many works have designed the IoV architecture based on vehicles and RSUs to realize the resource allocation in the IoV. The IoV architecture mentioned in [28] includes three communication methods: Vehicle-to-Vehicle (V2V), Vehicle-to-Infrastructure (V2I) and Infrastructure-to-Infrastructure (I2I). These communication methods are used to integrate the interaction between sensors and On-Board Units (OBU). The OBU provides computing power to vehicles to help drivers make decisions. Work [29] mentioned a software-defined social IoV networking model. The author realized green computing and communication based on the model. In [30], V2I and V2V communication methods were adopted to offload computing tasks to edge nodes by finding feasible paths. Work [31] proposed an IoV resource orchestration system with a three-layer architecture that generated top-down automatic resource orchestration configuration through prediction and update of edge and core network resources. In [32], the author thought that the Software-defined Internet of Vehicles (SDIoV) has emerged as a promising paradigm for next-generation vehicle resource management. Therefore, the author proposed an SDIoV framework based on vehicular cloud computing to enable vehicles to effectively enjoy computing services. To address the real-time response problem of vehicular fog computing (VFC), work [33] proposed a new IoV architecture. The allocation problem of limited resources is formulated by exploiting the real-time information interaction between parked vehicles and fog. The author in [34] divided the city IoV into different regions, and treated all edge servers in each region as a "big edge server", thereby reducing the impact of edge server distribution. Although V2X can make the communication in IoV more flexible, some problems are still to be

considered when applying them in computing offloading (e.g., trust problem and reward mechanism).

Computing offloading in the IoV is aimed at reducing costs, including energy consumption, delay, etc. Work [35] analyzed the impact of different task offloading decisions in the IoV on energy consumption, delay and packet loss rate. The author mentioned the computing offloading scheme in [36] to offload tasks on vehicles to edge computing devices. The difference from other works is that it considered the security problem in wireless communication. Work [37] analyzed the process of vehicle tasks and proposed the problem of vehicle task offloading. It formulated vehicle task offloading as a multi-objective optimization problem, which goal is to maintain load balance with the smallest total time cost. To meet users' demands while minimizing energy consumption, [38] proposed a green resource allocation mechanism. The mechanism calculated the optimal energy efficiency value by minimizing the energy consumption of wireless transmission. In [39], the author offloaded tasks from overloaded edge nodes to idle edge nodes. It proposed a multi-objective optimization problem aimed at minimizing energy consumption and delay. Work [40] proposed a wireless resource pre-allocation method in IoV, which predicted the resource allocation scheme through iterations considering different mobility modes and channel information. The authors in [41] considered the differences in delay tolerances of tasks and divided them into different priorities. Then, the author designed a task scheduling algorithm to adjust the order of tasks and allocate resources. In [42], the author proposed an IoV architecture based on fog computing and SDN, which had three communication modes: infrastructure and infrastructure (I2I) communication, vehicle-to-vehicle (V2V) communication and vehicle-to-infrastructure (V2I) communication. The concept of fog cluster is proposed to avoid frequent switching of data. Although the above works have achieved the allocation of computing resources, there has been no research on green heterogeneous computing powers allocation.

The establishment of the optimization model is the most important step in resource allocation, which determines the effectiveness of the resource allocation method. There are also many methods to solve optimization problems. The authors in [43] proposed the joint optimization problem of the total power of each RSU and the power distribution coefficient of IoV. The optimization problem was transformed and decoupled into two sub-problems using Dinkelbach's method, and the optimal solution of the optimization problem was obtained by using dual theory and KKT conditions. Work [44] modeled the resource allocation problem as a semi-Markov decision process in IoV. The joint optimization problem of computing and communication was modeled as a hierarchical architecture and used the hierarchical reinforcement learning method to obtain the optimal system resource allocation scheme. The green computing in [29] was modeled as an NP-hard optimization problem, which was solved by an adapted meta-heuristic solution Two-Way Particle Swarm Optimization (TWPSO) method. In [45], the author constructed computation, communication and interference models. To minimize the user utility

function, the Double Deep Q Network (DDQN) in deep reinforcement learning was used to solve the above optimization problem. The authors in [25] considered joint computing offloading and caching decisions and developed an online multi-decision scheme through the Lyapunov optimization method. In [46], the offloading decision of multiple tasks was modeled as a long-term planning problem. The author used a deep reinforcement learning method to obtain the best solution, and made decisions for the current tasks based on the knowledge learned from historical offloading decisions. The author optimized the allocation of communication and computing resources in work [47]. The optimization of offloading strategy had been realized through Q-learning method. In [48], the author proposed a distributed beyond-5G computing offload method. The computation offloading problem is formulated as a multi-agent Markov decision process. Then, the optimal resource orchestration results are obtained through the proposed online distributed reinforcement learning method. In the works we investigated, most of them used Lyapunov and RL to solve resource allocation problems, but Lyapunov can only achieve partial optimization. The advantage of reinforcement learning is that it is suitable for dynamic and highly complex scenarios. Although it requires a lot of training data, we can sample from historical training data for current training, thereby reducing the amount of data used.

Allocating heterogeneous resources reasonably to achieve green computing and communication is a challenging work due to the existence of heterogeneous computing power in IoV, but existing work has not conducted research on the above problem. Therefore, this paper introduces a green heterogeneous computing powers allocation method to reduce energy in the proposed HCP-IoV architecture in which we propose a long-term energy consumption optimization problem and use an improved reinforcement learning method to solve it.

### III. SYSTEM MODEL

#### A. System Structure

In order to realize the green allocation of heterogeneous computing power in IoV, we propose an HCP-IoV architecture based on SDN. The HCP-IoV architecture realizes the decoupling of network control and data transmission, which consists of three parts: RSUs, vehicles and SDN controller. As nodes in IoV, RSUs and vehicles contain heterogeneous computing powers: logic computing power, parallel computing power and neural network computing power.

In the proposed architecture, vehicles and RSUs collaborate to complete tasks based on their computing powers. We assume that there is only a single computing power on vehicles, while a variety of different computing powers are deployed on RSUs. To coordinate task offloading between vehicles and RSUs, we introduce SDN controller. As shown in Fig. 1, the SDN controller is connected to RSUs. Vehicles are connected to RSUs covering them and indirectly controlled by the SDN controller. There is data link and control link between vehicles and RSUs, only data link is between RSUs.

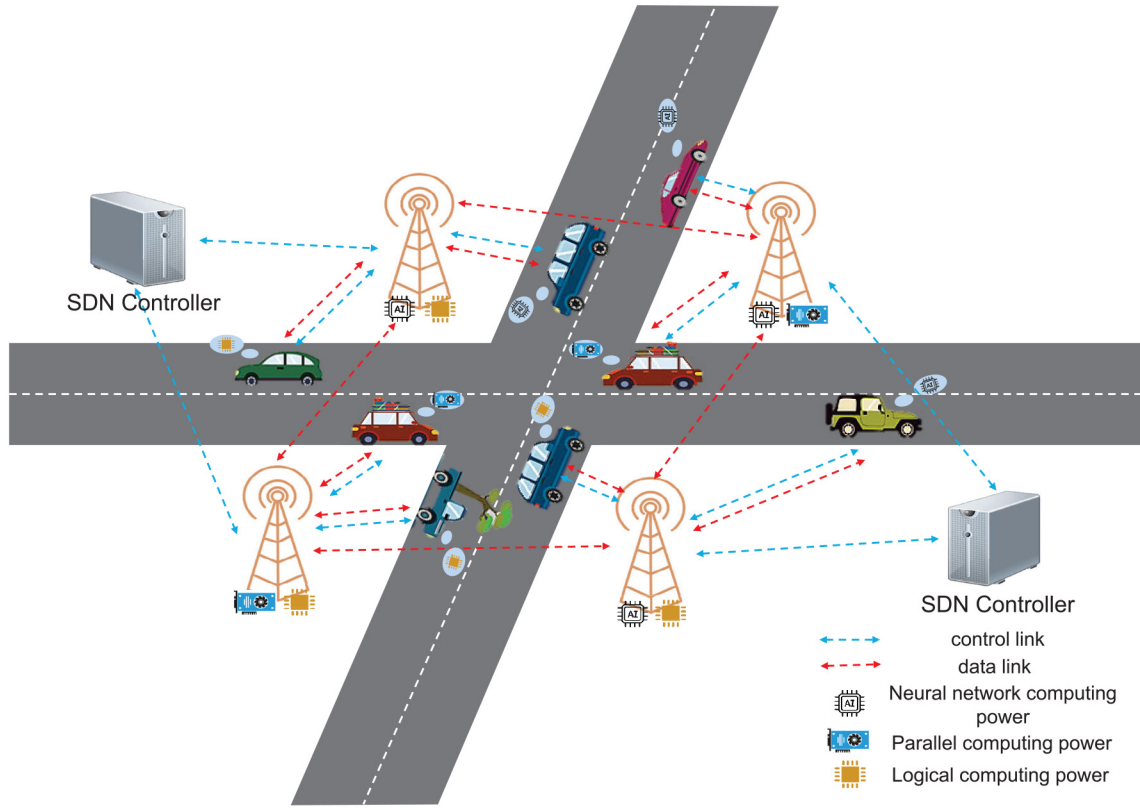


Fig. 1. SDN-based IoV architecture. Different computing resources are deployed on each RSU. The SDN controller controls global computing power resources. Vehicles can offload tasks to RSUs. Peer-to-peer offloading between RSUs. OFDM is used between vehicles and RSUs. The high-speed LAN is used between RSUs.

In this paper, the SDN controller enables collect computing power resources and computing requirements of the entire system in real time. Specifically, each RSU sends computing requirements to the SDN controller, after which the SDN controller allocates computing power resources according to the current network state (i.e., network resource conditions) to ensure latency and energy consumption requirements. RSUs and vehicles follow the OpenFlow protocol. OFDM [49] communication method is used between RSUs and vehicles, and a high-speed local area network [50] is adopted between RSUs.

### B. Communication Model

Based on the above architecture, we assume that there are  $M$  RSUs, denoted by  $\mathcal{M} = \{1, 2, \dots, M\}$ . Each RSU has a fixed coverage area. In the coverage area, vehicles can communicate with the RSUs and offload computing tasks to edge servers deployed in RSUs. They can offload computing tasks to RSUs that integrate edge servers. Moreover, we consider there are  $V$  vehicles overlaid by RSUs, denoted by  $\mathcal{V} = \{1, 2, \dots, V\}$ . We divide the continuous time  $T$  into discrete time slots, denoted by  $\mathcal{T} = \{0, 1, \dots, T-1\}$ . In each time slot, the SDN controller is aware of users' requirements and RSUs resources to make inferences and decisions. We consider two cases of computing offloading: 1) Vehicles can execute computing tasks locally or offload them to the edge server of RSU. 2) The edge server can execute computing tasks offloaded from vehicles overlaid by the RSU or offload these tasks to other edge

servers. Because of some advanced delay-sensitive applications, we only consider two-tier IoV architecture: vehicle layer and RSU layer. In addition, task offloading between RSUs can improve the efficiency of the system to complete tasks.

In this paper, we only consider uplink transmission delay. Because uploaded data size is much larger than downloaded data, the downlink transmission delay can be ignored. Without loss of generality, we assume that vehicle  $v$  arrives at the crossroad at a constant speed  $v_v$  in a fixed direction. Since vehicle  $v$  is in a different position in each time slot, 2-D coordinates of the vehicle  $v$  in time slot  $t$  is denoted by  $(x_v^t, y_v^t)$ . The coordinates of RSU  $m$  accessed by the vehicle  $v$  is  $(x_m^t, y_m^t)$ . So, the distance  $d$  between vehicle  $v$  and RSU  $m$  is:

$$d_{v,m}^t = \left[ (x_v^t - x_m^t)^2 + (y_v^t - y_m^t)^2 \right]^{\frac{1}{2}}, \quad (1)$$

According to Shannon Theory, the transmission rate between vehicle  $v$  and RSU  $m$  can be calculated by:

$$R_{v,m}^t = B \log_2 \left( 1 + \frac{p_v h_{v,m}^t (d_{v,m}^t)^{-\alpha}}{\sigma_{AWGN}^2} \right), \quad (2)$$

where  $B$  is the available channel bandwidth of the RSU  $m$ , and  $p_v$  is the transmission power of vehicle  $v$ . Moreover,  $\alpha$  is the path-loss exponent. From [26], the range of the pass-loss exponent is [3, 5], and  $h_{v,m}^t$  is the channel gain between RSU  $m$  and vehicle  $v$ .  $\sigma_{AWGN}^2$  is the power of additive white Gaussian noise (AWGN) [25]. Hence, the communication delay between



RSU  $m$  and its overlaid vehicles at time slot  $t$  is calculated by:

$$T_{t,m}^{com} = \sum_{v \in m_v^t} \frac{\tau_v^t}{R_{v,m}^t}, \quad (3)$$

where  $m_v^t$  is the set of vehicles overlaid by RSU  $m$  at time slot  $t$ , and  $\tau_v^t$  is the task size of vehicle  $v$  at time slot  $t$ . The energy consumption caused by task transmission on RSU  $m$  is denoted by:

$$E_{t,m}^T = \sum_{v \in m_v^t} p_v \bar{t}_v^t, \quad (4)$$

where  $p_v$  is the transmission power of vehicle  $v$ ,  $\bar{t}_v^t$  is the communication delay of vehicle  $v$  at time slot  $t$ . Here, we assume that the transmission power of vehicle  $v$  is a constant.

### C. Computation Model

For the above two computing offloading cases, we consider two offloading decisions which are vehicle-to-RSU and RSU-to-RSU offloading decisions. We assume that there are three types of computing tasks, which are logical computing tasks, parallel computing tasks and neural network computing tasks. The computing tasks need logical computing power, parallel computing power and neural network computing power to complete, which are integrated on edge servers. Generally, vehicles only have logical computing power, denoted by  $C_{V,L}$ . Without loss of generality, we assume that different edge servers deploy different sizes of computing power. The computing power on the edge server can be denoted by  $C_{m,L}$ ,  $C_{m,P}$  and  $C_{m,N}$ , which means logical computing power, parallel computing power and neural network computing power, respectively.

For the first offloading decision, vehicles can execute integral computing tasks locally or offload them integrally or partially to RSUs. We consider vehicle-to-RSU offloading decision consists of two parts: 1) vehicles can offload logical computing tasks to edge servers or not. 2) for other computing tasks, vehicles only offload them to edge servers. In the first part, we rewrite variable  $\tau_v^t$  as  $\tau_{v,L}^t$ . If vehicles decide to execute logical computing tasks locally, the computation delay can be calculated by:

$$T_{t,v}^{Loc} = \frac{(1 - x_v^t) \tau_{v,L}^t f_L}{C_{V,L}}, \quad (5)$$

where  $x_v^t$  is the offload decision of vehicle  $v$ , in which range is  $[0, 1]$ . If  $x_v^t = 0$ , vehicle  $v$  decides to execute integrally computing task locally. If  $x_v^t = 1$ , vehicle  $v$  decides to offload integrally computing task to edge server. If  $x_v^t$  is between 0 and 1, vehicle  $v$  decides to offload computing task to edge server partially.  $f_L$  is the amount of logical computing power needed to calculate 1-bit task.  $C_{V,L}$  is the logical computing power of each vehicle. In the second part, vehicles can't execute parallel computing and neural network computing tasks. So, vehicles can only offload tasks to edge servers. Then the energy consumption of computing tasks on vehicle  $v$  is:

$$E_{t,v}^{Loc} = \gamma_L (1 - x_v^t) \tau_{v,L}^t, \quad (6)$$

where  $\gamma_L$  is the computing energy consumption per bit task of vehicle  $v$ .

For the second offloading decision, RSU  $m$  can offload computing tasks integrally or partially to the other RSUs. For a certain type of task, we assume that there are three kinds of RSUs: 1) only offload tasks and don't receive tasks. 2) only receive tasks and don't offload tasks. 3) neither offload tasks nor receive tasks. Generally, this is in line with the actual situation because when RSUs offload tasks, its computing power is usually not enough to complete tasks. The task of the system in time slot  $t$  is represented as  $\Phi_t$ . We denote  $\psi_{m,i}^t$  and  $\varphi_{m,i}^t$  ( $i \in \{L, P, N\}$ , which represent different tasks.) as the total tasks and tasks offloaded from vehicles on RSU  $m$  at time slot  $t$ , respectively. Then the tasks of the system are represented by the following equation:

$$\Phi_t = \sum_{m,i} \varphi_{m,i}^t + \sum_v (1 - x_v^t) \tau_{v,L}^t. \quad (7)$$

The total tasks on RSU  $m$  at time slot  $t$  is divided into two parts, one is offloaded from vehicles, denoted by  $\varrho_{m,i}^t$ , and the other is offloaded from other RSUs, denoted by  $\rho_{m,i}^t$ . Here,  $\psi_{m,i}^t$  should satisfies  $\psi_{m,i}^t = \varrho_{m,i}^t + \rho_{m,i}^t$ . When  $\rho_{m,i}^t = 0$  and  $\varrho_{m,i}^t < \varphi_{m,i}^t$ , RSU  $m$  belongs to the first type of RSU. When  $\rho_{m,i}^t = 0$  and  $\varrho_{m,i}^t = \varphi_{m,i}^t$ , RSU  $m$  belongs to the third type of RSU. When  $\rho_{m,i}^t > 0$  and  $\varrho_{m,i}^t = \varphi_{m,i}^t$ , RSU  $m$  belongs to the second type of RSU. We assume that  $y_{mn,i}^t$  represents the offloading decision from RSU  $m$  to RSU  $n$  at time slot  $t$  for task  $i$ . Then, the tasks offloaded in RSU  $m$  from other RSUs and vehicles can be expressed as:

$$\rho_{m,i}^t = \sum_{n \in \mathcal{M}, n \neq m} y_{nm,i}^t \varphi_{n,i}^t, \quad (8)$$

$$\varrho_{m,i}^t = \left( 1 - \sum_{n \in \mathcal{M}, n \neq m} y_{mn,i}^t \right) \varphi_{m,i}^t. \quad (9)$$

Then the computation delay of RSU  $m$  is calculated by:

$$T_m^t = \sum_i \frac{\psi_{m,i}^t}{C_{m,i}}. \quad (10)$$

Therefore, the total computation delay between RSUs is given by:

$$T_t^{RSU} = \sum_m \sum_i \frac{\psi_{m,i}^t}{C_{m,i}}. \quad (11)$$

Finally, the total energy consumption is:

$$E_t^{RSU} = \sum_m \sum_i \gamma_{R,i} \psi_{m,i}^t, \quad (12)$$

where  $\gamma_{R,i}$  is the energy consumption per bit task  $i$  on RSUs.

### D. Problem Formulation

Considering that there are limited and multiple computing powers on RSUs, the appropriate computing powers are allocated to vehicles according to their demands can minimize system delay. In each time slot  $t$ , the system delay consists of two parts: transmission delay and computing delay. The

offloading decisions of vehicles are added to (3), which is rewritten as:

$$T_{t,m}^{com} = \sum_{v \in m_v^t} \frac{x_v^t \tau_v^t}{R_{v,m}^t}. \quad (13)$$

The system delay and energy consumption are denoted by:

$$T_t^{Sys} = \sum_m T_{t,m}^{com} + \sum_v T_{t,v}^{Loc} + T_t^{RSU}, \quad (14)$$

$$E_t^{Sys} = \sum_m E_{t,m}^T + \sum_v E_{t,v}^{Loc} + E_t^{RSU}. \quad (15)$$

$\mathbf{x}$  and  $\mathbf{y}$  are denoted as the offloading decisions of vehicles and RSUs, respectively. We denote the latency of computing one task as:

$$T_{t,v} = T_{t,v}^{Loc} + \frac{x_v^t \tau_v^t}{R_{v,m}^t} + \frac{\left(1 - \sum_{n \in \mathcal{M}, n \neq m} y_{mn}^t\right) x_v^t \tau_v^t}{C_m} + \frac{y_{n_1}^t x_v^t \tau_v^t}{C_{n_1}} + \dots \quad (16)$$

where the fourth item and later represent the computation delay of offloading tasks to other RSUs.

The optimization problem is as follows:

$$\min_{\mathbf{x}, \mathbf{y}} \sum_{t=0}^{T-1} E_t^{Sys}, \quad (17)$$

$$\text{s.t. } T_{t,v} \leq T_{t,v}^{max}, \quad \forall t \in \mathcal{T}, \forall v \in \mathcal{V}, \quad (2a)$$

$$E_t^{Sys} \leq E_{max}, \quad \forall t \in \mathcal{T}, \quad (2b)$$

$$x_v^t \in [0, 1], \forall v \in \mathcal{V}, \quad \forall t \in \mathcal{T}, \quad (2c)$$

$$y_{mn,i}^t \in [0, 1], \forall m, n \in \mathcal{M}, \quad \forall t \in \mathcal{T}, \quad \forall i \in \{L, N, P\}, \quad (2d)$$

where  $T_{t,v}^{max}$  represents the maximum delay that can be tolerated to complete computing tasks.  $E_{max}$  represents the maximum energy that can be consumed to complete computing tasks. Constraint (16a) indicates that the total delay (communication and computation) in completing the task cannot exceed the maximum delay that the computing task can tolerate. Constraint (16b) indicates that the total energy consumption (communication and computation) to complete the task cannot exceed the maximum energy that can be consumed to complete the task.

The optimization problem needs to find the best vehicles and RSUs offloading decisions for all types of tasks to minimize long-term energy consumption to achieve green computing. Here, the number of vehicles and RSUs and the continuity of their offloading leads to a huge solution space of the optimization problem. Although there are some constraints, it is still challenging to find the optimal solution to the optimization problem in such a huge solution space.

#### IV. PROBLEM SOLUTION

In this section, we propose a hierarchical resource allocation method based on reinforcement learning to solve the optimization problem in Section IV-C.

We assume that vehicles and RSUs make offloading decisions at the beginning of the time slot. All tasks are completed at the end of the time slot. The SDN controller awares network resources at the beginning of the time slot. Then, the SDN controller gives the best offloading decisions for vehicles and RSUs. Here, the decisions are made hierarchically.

To ensure task delay and minimize the energy consumption of the system, we formulate the computing power allocation problem as a Markov decision process (MDP) [51]. The formulated MDP is represented by  $\mathbf{M} = (\mathcal{S}, \mathcal{A}, \mathbf{P}, \mathbf{R}, \gamma)$ , where  $\mathcal{S}, \mathcal{A}$  are the set of states and actions,  $\mathbf{P}$  is state transition matrix,  $\mathbf{R}$  is reward function and  $\gamma$  is discount factor. In the computing power allocation problem, we consider the HCP-IoV as the environment. The SDN controller makes the best hierarchical offloading decisions by awaring the resources in the environment. We assume that different types of computing tasks have the same state and action space, but different units and values. We regard the channel as a finite-state Markov channel. More details on MDP are given in the following subsections.

##### A. State

We use discretized channel gain as the state of the Markov chain, which can be given as:

$$\mathcal{S} = \{s^t | s^t = (h_{v,m})\}, \forall v \in \mathcal{V}, \forall m \in \mathcal{M}, \quad (18)$$

where  $s$  indicates the channel gain between RSU  $m$  and vehicle  $v$  at time slot  $t$  in the system, which obeys Markov property. The channel gain at the current moment is related to the previous moment. The state transition is realized through state transition matrix  $\mathbf{P}$ .

##### B. Action

Reinforcement learning method uses a parameterized policy  $\pi_\theta$  to obtain action  $\mathbf{a}$  based on the current state. The action at each time slot  $t$  is denoted by:

$$\mathcal{A} = \{\mathbf{a}^t = (\mathbf{x}, \mathbf{y}) | \mathbf{a}^t = (x_v, \mathbf{y}_{mn,L}, \mathbf{y}_{mn,P}, \mathbf{y}_{mn,N})\}, \quad \forall v \in \mathcal{V}, \forall m, n \in \mathcal{M}. \quad (19)$$

where  $\mathbf{a}^t$  is a hierarchical action, as shown in Fig. 2. It contains computing offloading decisions of vehicles and RSUs for different computing powers. The dimensionality of action  $\mathbf{a}^t$  depends on the number of vehicles and RSUs. The dimension of the action is proportional to the number of vehicles, RSUs, and computing powers in the system.

##### C. Reward

We use the reciprocal of the optimization goal as the reward. The optimization goal of this paper is to minimize the long-term energy consumption of the system, represented by:

$$r = \frac{1}{E^{Sys}}. \quad (20)$$

The equation shows that the reward is higher, while the energy consumption of the system is smaller. We use reinforcement

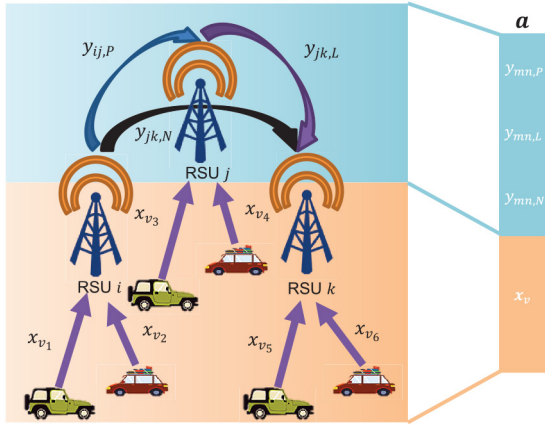


Fig. 2. Hierarchical offloading decision.

learning to maximize long-term rewards to find hierarchical offloading decisions that minimize the long-term energy consumption of the system.

Given a state  $s$ , the agent gets an immediate reward  $r(s, a)$  by performing an action  $a$ . The action is executed in per time slot  $t$ . Return  $\mathcal{O}$  is the cumulative sum of long-term rewards. The cumulative reward for time slot  $t$  is:

$$\mathcal{O}_t = r_{t+1} + \Upsilon r_{t+2} + \Upsilon^2 r_{t+3} + \dots, \quad (21)$$

which can also be expressed as:

$$\mathcal{O}_t = \sum_{i=t}^{\infty} \Upsilon^{i-t} r_{i+1}, \quad (22)$$

where  $\Upsilon$  is discount factor, and its range is  $(0, 1]$ .  $t$  represents the current time slot and  $i$  represents the  $i$ -th time slot.

#### D. MGD-PPO Method

The optimal solution to the MDP problem is to find the strategy  $\pi^*$  that maximizes the return. Under state  $s$ , an action  $a$  is selected according to a probability distribution, which is expressed as  $\pi(a|s)$ .

We adopt a reinforcement learning method based on policy gradient to solve the action continuity in the proposed MDP problem. The hierarchical resource allocation structure and multi-dimensional problem of actions in this paper need to improve the existing reinforcement learning methods. Hence, we use multivariate Gaussian distribution to improve the PPO method. The PPO method is an improvement of the Policy Gradient (PG) method.

The improved PPO method uses Actor-Critic (AC) model. We use parameterized neural networks to represent strategies  $\pi_\theta$ . This parameterized neural network is the actor network. The expected return can be maximized by adjusting parameter  $\theta$ . The expected return is represented by:

$$\mathcal{J}(\theta) = \mathbb{E}_{\xi \sim p(\xi|\theta)} [\mathcal{O}(\xi)], \quad (23)$$

where  $\xi$  is a complete path for the agent to explore the optimal solution of the resource allocation problem from the beginning to the end and  $p_\theta$  is the probability of the path. The gradient

of  $\mathcal{J}(\theta)$  is:

$$\begin{aligned} \nabla_\theta \mathcal{J} &= \sum_{\xi} \mathcal{O}(\xi) \nabla_\theta p(\xi | \theta) \\ &= \sum_{\xi} \mathcal{O}(\xi) p(\xi | \theta) \frac{\nabla_\theta p(\xi | \theta)}{p(\xi | \theta)} \\ &= \sum_{\xi} \mathcal{O}(\xi) p(\xi | \theta) \nabla_\theta \log p(\xi | \theta) \\ &\approx \frac{1}{N} \sum_{n=1}^N \mathcal{O}(\xi^n) \nabla_\theta \log p(\xi^n | \theta), \end{aligned} \quad (24)$$

$p(\xi^n | \theta)$  is denoted by:

$$\begin{aligned} p(\xi^n | \theta) &= p(s^1) p(a^1 | s^1, \theta) p(r^1, s^2 | s^1, a^1) p(a^2 | s^2, \theta) \\ &\quad \dots p(a^t | s^t, \theta) p(r^t, s^{t+1} | s^t, a^t) \\ &= p(s^1) \prod_t p(a^t | s^t, \theta) p(r^t, s^{t+1} | s^t, a^t), \end{aligned} \quad (25)$$

where  $p(a^t | s^t, \theta)$  is obtained from a probability distribution.

The probability distribution is composed of variance and mean from the actor network. Due to the large dimension of actions, it is necessary to output multiple means and covariance matrix from the actor network to form a Multivariate Gaussian Distribution (MGD). When the input of the actor network is  $s^t$ , the probability density function of MGD is:

$$p(a_1^t, a_2^t, \dots, a_k^t) = \frac{1}{(2\pi)^{\frac{k}{2}} |\Sigma_t|^{\frac{1}{2}}} e^{-\frac{1}{2} [(a^t - \mu^t)^\top \Sigma_t^{-1} (a^t - \mu^t)]}, \quad (26)$$

where  $\Sigma_t$  is covariance matrix and  $\mu^t$  is mean vector.  $p(a^t | s^t, \theta)$  is obtained from  $p(a_1^t, a_2^t, \dots, a_k^t)$ .

We integrate (25) into  $\log p(\xi^n | \theta)$ , then obtain the equation:

$$\begin{aligned} \log p(\xi^n | \theta) &= \log \left( p(s^1) \prod_t p(a^t | s^t, \theta) \right. \\ &\quad \left. \times p(r^t, s^{t+1} | s^t, a^t) \right) \\ &= \log p(s^1) + \sum_{t=1}^T \log p(a^t | s^t, \theta) \\ &\quad + \sum_{t=1}^T \log p(r^t, s^{t+1} | s^t, a^t). \end{aligned} \quad (27)$$

Since the first and third terms of the above formula do not have the parameter  $\theta$ , the gradient of the two terms is 0. Then, we can obtain:

$$\nabla_\theta \log p(\xi^n | \theta) = \sum_{t=1}^T \nabla_\theta \log p(a^t | s^t, \theta). \quad (28)$$

Then,  $\nabla_\theta \mathcal{J}$  is:

$$\nabla_\theta \mathcal{J} \approx \frac{1}{N} \sum_{n=1}^N \mathcal{O}(\xi^n) \nabla_\theta \log P(\tau^n | \theta)$$

$$\begin{aligned}
&= \frac{1}{N} \sum_{n=1}^N \mathcal{O}(\xi^n) \sum_{t=1}^T \nabla_{\theta} \log p(\mathbf{a}^t | \mathbf{s}^t, \theta) \\
&= \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \mathcal{O}(\xi^n) \nabla_{\theta} \log p(\mathbf{a}^t | \mathbf{s}^t, \theta), \quad (29)
\end{aligned}$$

where  $T_n$  is the number of explorations in each episode. We adopt advantage function to weigh the impact of reward at different moments on the current moment while reducing the variance of return allocation. The estimation of the advantage function of PPO method is defined as:

$$\begin{aligned}
\hat{A}(\mathbf{s}^t, \mathbf{a}^t) &= \zeta_t + (\mathbf{\Upsilon}\lambda)\zeta_{t+1} + \dots \\
&\quad + \dots + (\mathbf{\Upsilon}\lambda)^{T-t+1}\zeta_{T-1}, \quad (30)
\end{aligned}$$

where  $\zeta_t = \mathbf{r}^t + \mathbf{\Upsilon} \mathbf{V}(\mathbf{s}^{t+1}) - \mathbf{V}(\mathbf{s}^t)$  and  $\lambda$  is the weight parameter. The advantage function of different estimates can be obtained by adjusting  $\lambda$ . In this paper, we denote  $\lambda$  as 1.  $\mathbf{V}(\mathbf{s}^t)$  is value function. It is obtained from critic network.

Replacing  $\mathcal{O}(\xi^n)$  in (26) with the advantage function, we get the following result:

$$\nabla_{\theta} \mathcal{J} = \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \hat{A}(\mathbf{s}^t, \mathbf{a}^t) \nabla_{\theta} \log p(\mathbf{a}^t | \mathbf{s}^t, \theta). \quad (31)$$

The PPO method improves the training speed by introducing importance sampling. It improves new strategies by sampling old strategies. Then, the improved gradient is:

$$\begin{aligned}
\nabla_{\theta} \mathcal{J} &= \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \frac{p_{\theta}(\mathbf{a}^t | \mathbf{s}^t)}{p_{\theta'}(\mathbf{a}^t | \mathbf{s}^t)} \\
&\quad \hat{A}(\mathbf{s}^t, \mathbf{a}^t) \nabla_{\theta} \log p(\mathbf{a}^t | \mathbf{s}^t, \theta), \quad (32)
\end{aligned}$$

To prevent  $\theta$  and  $\theta'$  from being too different, PPO adds the clipping method to likelihood function. The likelihood function is:

$$\begin{aligned}
\mathcal{J}(\theta) &= \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \min \left( \frac{p_{\theta}(\mathbf{a}^t | \mathbf{s}^t)}{p_{\theta'}(\mathbf{a}^t | \mathbf{s}^t)} \hat{A}(\mathbf{s}^t, \mathbf{a}^t), \right. \\
&\quad \left. clip \left( \frac{p_{\theta}(\mathbf{a}^t | \mathbf{s}^t)}{p_{\theta'}(\mathbf{a}^t | \mathbf{s}^t)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}(\mathbf{s}^t, \mathbf{a}^t) \right), \quad (33)
\end{aligned}$$

where  $\epsilon$  is a hyper-parameter.

We input the channel gain into the actor network and the critic network. Mean values of MGD are generated from the actor network. A covariance matrix is initialized and added to the learnable parameters. The MGD of the multi-dimensional offloading decision can be obtained from the initialized covariance matrix and the mean values generated by the actor network. We sample offloading decisions based on MGD. Then, the channel gain and the offloading decision obtained by sampling are put into our model. The offloading decision is executed hierarchically in our model, which is executed at the vehicle level first and then at the RSU level. The reward value is returned and stored after calculating the delay and energy consumption according to the model. Repeat the above process until the update conditions are met. We substitute the offloading decision probability and the saved rewards into (33) to

---

#### Algorithm 1 Heterogeneous Computing Power Allocation Algorithm

---

**Require:** Initialize the actor network with random parameters  $\theta$ . Initialize the critic network with random parameters  $\theta_v$ . Initialize state  $\mathbf{s}^t$ . Initialize the covariance matrix  $\Sigma_1$ .

**Ensure:** Optimal green heterogeneous computing power allocation decision

```

1: for  $t \leftarrow 1, \dots, T$  do
2:   Input  $\mathbf{s}^t$  into the actor network
3:   Output the mean value  $\boldsymbol{\mu}^t$  from the actor network
4:   Substitute  $\Sigma_t$  and  $\boldsymbol{\mu}^t$  into (26) to get the probability density function  $p(a_1^t, a_2^t, \dots, a_k^t)$ 
5:   Sampling according to the probability density function to get the action  $\mathbf{a}_t$ 
6:   Substitute actions into (5) and (6)
7:   if  $y_{nm,i}^t \neq 0$  and  $y_{mn,i}^t \neq 0$  then
8:      $y_{nm,i}^t \leftarrow 0, y_{mn,i}^t \leftarrow 0$ 
9:   end if
10:  Substitute the updated action into (8) and (9)
11:  Calculate  $T_t^{Sys}$  and  $E_t^{Sys}$  according to (14) and (15)
12:  if  $T_t^{Sys} \leq T_{max}$  and  $E_t^{Sys} \leq E_{max}$  then
13:    Calculate  $\mathbf{r}$  according to (20)
14:  else
15:     $\mathbf{r} \leftarrow 0$ 
16:  end if
17:   $\mathbf{s}^t \xrightarrow{P} \mathbf{s}^{t+1}$ 
18:  if  $t = BATCH\_SIZE$  then
19:    Calculate  $\hat{A}(\mathbf{s}^t, \mathbf{a}^t)$  according to (30)
20:    Calculate loss function  $\mathcal{J}(\theta)$  according to (33)
21:    Update  $\theta \leftarrow \theta - \nabla_{\theta} \mathcal{J}(\theta)$ 
22:    Update  $\theta_v \leftarrow \theta_v - \nabla_{\theta_v} \mathcal{J}(\theta_v)$ 
23:  end if
24:   $\mathbf{s}^t \leftarrow \mathbf{s}^{t+1}$ 
25: end for

```

---

calculate the loss function. According to the gradient obtained in (32), the parameters of the actor network and the critic network are updated.

Repeat the above process to get the optimal solution to the optimization problem finally. The specific flow of the algorithm is shown in Algorithm 1.

## V. PERFORMANCE EVALUATION

In this section, we evaluate the performance of the proposed method. We simulated the scheduling of heterogeneous computing powers in a crossroad area, where two RSUs are distributed near the crossroad to provide computing powers for vehicles. These two RSUs have different computing powers. RSU and vehicle nodes are distributed in a 300m  $\times$  300m area, where the coverage of each RSU is 100m. All vehicles in the area can only be connected to one RSU until it leaves the coverage of the RSU that it is connected to. The logical computing power of vehicles and RSUs is measured by CPU cycles. The single-precision calculation rate measures the neural network and parallel computing power on RSU. The computing power required to calculate each bit of logical task



TABLE I  
SIMULATION PARAMETERS

| Parameters                                        | Value    |
|---------------------------------------------------|----------|
| The transmission power of vehicles, $p_v$         | 100mW    |
| The range of motion of vehicles, $d_{v,m}$        | [10,50]m |
| Bandwidth between each vehicle and RSU, $B$       | 2GHz     |
| Logical computing power of RSUs, $C_{m,L}$        | 60GHz    |
| Logical computing power of vehicles, $C_{V,L}$    | 20GHz    |
| Parallel computing power of RSUs, $C_{V,P}$       | 60GHz    |
| Neural network computing power of RSUs, $C_{V,N}$ | 60GHz    |
| Logical computing task, $\tau_{v,L}$              | 30Mb     |
| Parallel computing task, $\tau_{v,P}$             | 300Mb    |
| Neural network computing task, $\tau_{v,N}$       | 300Mb    |
| Maximum delay requirement, $T_{t,v}^{max}$        | 0.59s    |

is 1 cycle. The computing power required to calculate each bit of parallel and neural network task is 0.08 cycle. The discount factor of the proposed method is set to 0.9. We set the learning rates of the actor network and critic network to 0.0001 and 0.002, respectively. The batchsize is set to 100. Some other important parameters are given in Table I [22], [25], [27]. In reality, the speed of vehicles is not controllable. Therefore, to consider the mobility of the vehicle, we convert the fixed vehicles' speed into a varying distance  $d_{v,m}$  between vehicles and RSUs, which is shown in Table I. We set the range of motion of vehicles  $d_{v,m}$  is [10, 50]m. In the process of each computing power allocation, the distance between vehicles and RSUs is different, which meets the condition that the vehicle is not controllable in reality.

We compare our method with five other methods, including:

(a) *No heterogeneous computing power resource allocation (NoHCP-MGDPPO)*: This method does not consider the allocation of heterogeneous computing powers. There is no division of heterogeneous computing powers between vehicles and RSUs.

(b) *Random allocation of heterogeneous computing powers (HCP-Random)*: Reinforcement learning method are not used here to find the optimal solution. The offloading decisions of vehicles and RSUs are random.

(c) *Local calculation (Local)*: Vehicles only computes tasks locally and do not offload tasks to RSUs.

(d) *RSU-No heterogeneous computing power resource allocation (NoHCP-RSU-MGDPPO)*: Vehicles do not compute locally. They offload all tasks to RSUs and there is no heterogeneous computing powers allocation between RSUs.

(e) *RSU-Heterogeneous computing power resource allocation (HCP-RSU-MGDPPO)*: Vehicles offload all tasks to RSUs and there has heterogeneous computing power allocation between RSUs.

(f) *Heterogeneous computing power resource allocation with Q-learning (HCP-Q-learning)*: The Q-learning method [47] is used to allocate heterogeneous computing power resources. The action space of the proposed method is continuous, but Q-learning only adapts to discrete action space. Therefore, we discretize the action space. The

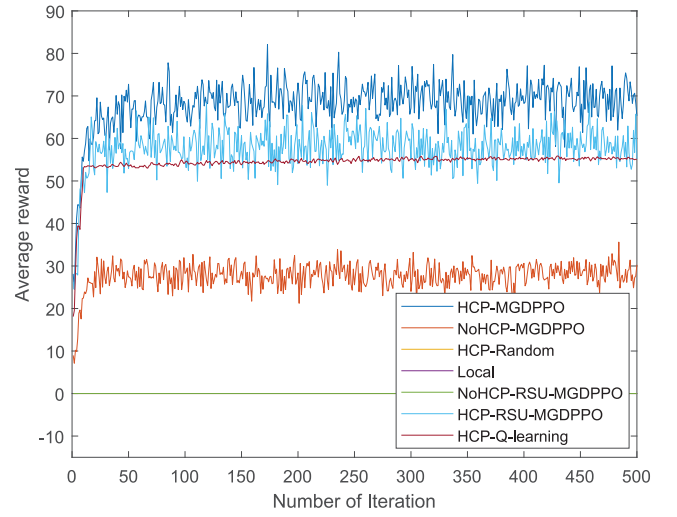


Fig. 3. Average reward comparison of different methods.

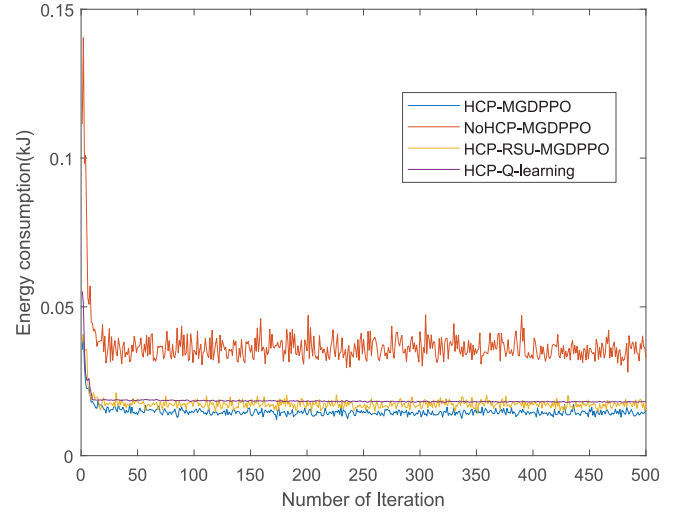


Fig. 4. Energy consumption comparison of different methods.

offloading decisions of vehicles and RSUs are discretized as  $[0, 0.1, 0.2, \dots, 1]$ .

#### A. System Performance

Fig. 3 and Fig. 4 show the performance of different methods. The horizontal axis of Fig. 3 and Fig. 4 represents the number of iterations of the method. The vertical axis represents the average reward and energy consumption of the method, respectively. It can be seen from the figure that our method gets the highest reward, which means that our method has the lowest long-term energy consumption under the constraint of delay. The second highest reward in the figure is the method (e). Since it uses heterogeneous computing powers allocation method, its energy consumption is lower than other methods, although it does not use local computing. On the other hand, it can also be seen that local computing also contributes to energy consumption. Since method (a) does not use heterogeneous computing power allocation, its average reward (i.e., energy consumption) is far lower than the method using heterogeneous computing power allocation. The rewards of

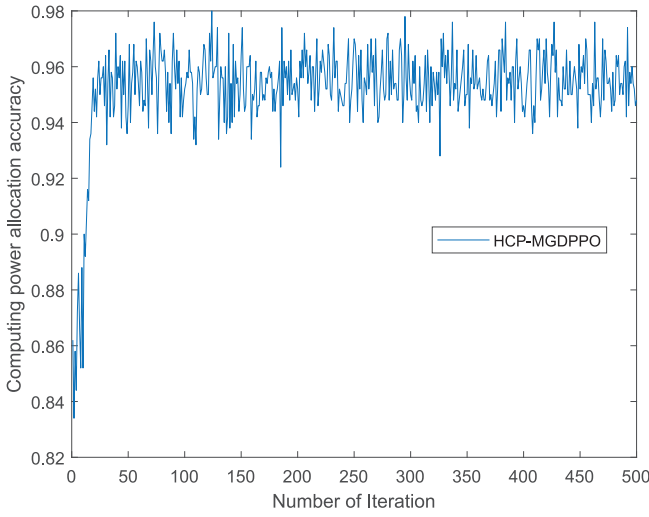


Fig. 5. The accuracy of the computing power allocation of our method.

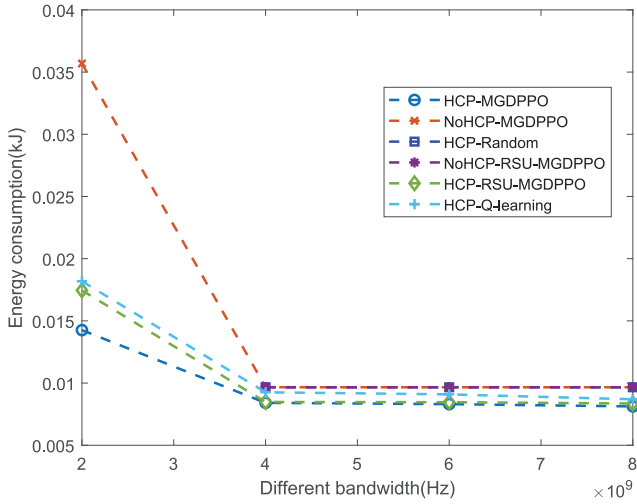


Fig. 6. The impact of different bandwidths.

methods (b), (c), and (d) overlap on the same line in Fig. 3. The reason why the reward overlaps to 0 is that we set a delay constraint. The offloading decisions rewards that do not meet the delay constraint are set to 0. According to our simulation results, the above three methods do not meet our delay requirements. The delay of method (b) is more than 0.75s. The delay of methods (c) and (d) is 0.736s and 0.6s, respectively. The reason why the above three methods are not shown in Fig. 4 is that they do not meet the delay constraint.

Fig. 5 shows that as the number of iterations increases, the accuracy of our method gradually increases. In the initial stage of allocating computing power, our method can achieve an accuracy rate of 82%. The accuracy of our method can reach up to 98%, which is close to 100%. It can meet the requirements of users.

### B. Influence of Parameters

In this subsection, we evaluated the impact of parameters on all methods. Fig. 6 shows the impact of different bandwidths on all methods. We set the range of bandwidth to [2, 8] GHz.

TABLE II  
THE IMPACT OF DIFFERENT VEHICLES

| Number of vehicles | Delay | Convergence episodes |
|--------------------|-------|----------------------|
| 4                  | 2.35s | 58                   |
| 6                  | 3.55s | 48                   |
| 8                  | 4.7s  | 52                   |
| 10                 | 5.9s  | 52                   |

It can be seen from the figure that the energy consumption of most methods will decrease with the increase of bandwidth. Comparing several methods in the figure, the energy consumption of our proposed method is the lowest. The energy consumption between our proposed method and method (e) is very close because as the bandwidth increases, the transmission energy consumption will decrease. In this way, the energy consumption of transmission imposes less and less constraint on the offloading of vehicles to RSUs. As a result, vehicles will offload a large number of tasks to RSUs. Although the energy consumption of method (f) decreases with increasing bandwidth, the computational task consumes more energy than our proposed method. It is not difficult to see from the figure that all methods have inflection points. The reason is that as the bandwidth increases, most of the methods meet the delay constraint. Transmission bandwidth no longer has a significant impact on energy consumption. Methods (a), (b) and (d) overlap after the bandwidth is greater than or equal to 4 GHz. This shows that when the bandwidth is greater than or equal to 4 GHz, vehicles will offload all tasks to RSUs. At this time, the increase in bandwidth has no effect on system energy consumption at all. The reason we do not put method (c) in the figure is that its reward is 0, which means no matter how the bandwidth increases, the local computation method cannot meet the delay requirement.

Table II shows the influence of different vehicle numbers on the method. We set the number of vehicles to 4, 6, 8, 10, respectively. We compared the delay and the speed of method convergence of different numbers of vehicles. It can be seen from the table that as the number of vehicles increases, the delay also increases. This is in line with the actual situation. Because when vehicles increase and resources are limited, the system delay will inevitably increase. We also compared the convergence speed of the method with different numbers of vehicles. Because the increase in the number of vehicles will increase the dimensionality of the action, which may affect the convergence speed of the method. It can be seen from the table that the convergence speed of our proposed method does not increase with the increasing number of vehicles, which also proves the robustness of the method.

Fig. 7 shows the impact of different vehicles computing power on energy consumption. We set the range of vehicle computing power as [2, 5] GHz. It can be seen from the figure that the energy consumption will decrease as the vehicles computing power increases. Our proposed method still performs best among all methods. There is a crossover between method (a) and method (e), which is because there is no local

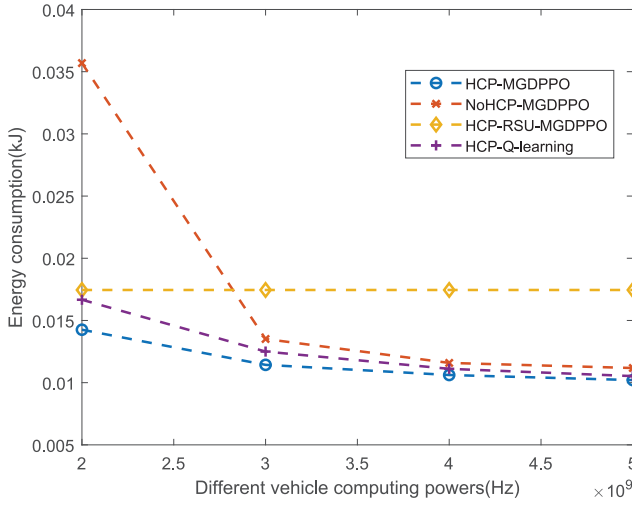


Fig. 7. The impact of different vehicle computing powers.

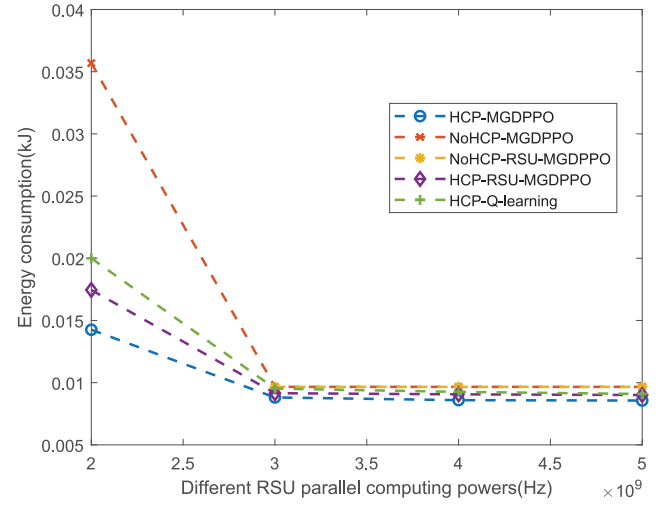


Fig. 9. The impact of different RSU parallel computing power.

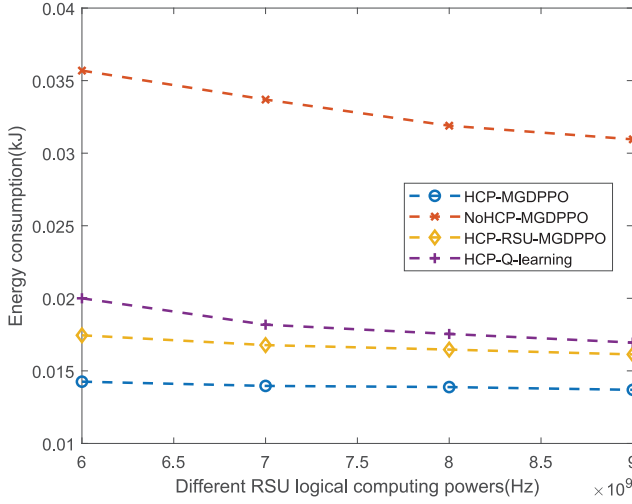


Fig. 8. The impact of different RSU logical computing power.

computation in method (e). The increase in vehicle computing power only contributes to method (a), (f) and our proposed method but does not effect method (e). The rewards of methods (b), (c), and (d) that we do not show in the figure are 0. This does not mean that the increase in vehicles' computing power has no impact on these three methods. According to our experimental results, the increase in vehicle computing power has an impact on method (c). Corresponding to different computing powers of vehicles, the delay of method (c) are 0.368s, 0.365s, 0.364s, 0.363s. Although it does not meet the delay requirement, its delay is reduced.

We compare the impact of different RSU's logical computing power on system energy consumption in Fig. 8. We set the interval of logical computing power as [6, 8] GHz. It can be seen from the figure that the energy consumption of all methods decreases with the increase of logical computing power. The reason for the situation is that when the computing power increases, the time required for the computation task is reduced. Thereby, the system energy consumption is correspondingly reduced. The proposed method by us still has

the lowest energy consumption among all methods. Methods (b), (c), and (d) are not shown in Fig. 8 because their delays do not meet the delay requirement.

In Fig. 9, we set up different RSU parallel computing power, which interval is [2, 5] GHz. It can be seen from the figure, the energy consumption of our method is the smallest. All the methods in the figure have an inflection point because when the parallel computing power is increased to 3 GHz, all the methods meet the delay requirements. Any further increase in parallel computing power has no impact on all methods. The energy consumption of methods (b) and (d) becomes flat after the parallel computing power is 3 GHz, which shows that the best decisions of these two methods are to offload all tasks to RSUs.

## VI. CONCLUSION

In this paper, we build an HCP-IoV architecture, which consists of RSUs, vehicles and SDN controller. The green allocation scheme of IoV heterogeneous computing powers is realized based on the HCP-IoV architecture to jointly optimize communication and computing power problem. To address this optimization problem, We explore the optimal strategy for heterogeneous computing power allocation by improving the PPO algorithm, thereby reducing the delay and energy consumption of the HCP-IoV. Finally, we verify the effectiveness of the proposed method through simulation results.

## REFERENCES

- [1] X. Pei, H. Yu, X. Wang, Y. Chen, M. Wen, and Y.-C. Wu, "NOMA-based pervasive edge computing: Secure power allocation for IoV," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 5021–5030, Jul. 2021.
- [2] P. Dong, Z. Ning, R. Ma, X. Wang, X. Hu, and B. Hu, "NOMA-based energy-efficient task scheduling in vehicular edge computing networks: A self-imitation learning-based approach," *China Commun.*, vol. 17, no. 11, pp. 1–11, Nov. 2020.
- [3] L.-B. Bilius and R.-D. Vatavu, "A synopsis of input modalities for in-vehicle infotainment and consumption of interactive media," in *Proc. ACM Int. Conf. Interact. Media Exp.*, 2020, pp. 195–199.
- [4] Z. Lv, J. Lloret, and H. Song, "Guest editorial software defined Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 6, pp. 3504–3510, Jun. 2021.

- [5] N. Cvijetic and T. Tomazin, "Developing a centralized compute architecture for autonomous vehicles," *ATZelectronics Worldwide*, vol. 16, no. 1, pp. 10–15, 2021.
- [6] X. Tang *et al.*, "Computing power network: The architecture of convergence of computing and networking towards 6G requirement," *China Commun.*, vol. 18, no. 2, pp. 175–185, Feb. 2021.
- [7] T. Qiu, J. Chi, X. Zhou, Z. Ning, M. Atiqzaman, and D. O. Wu, "Edge computing in Industrial Internet of Things: architecture, advances and challenges," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 4, pp. 2462–2488, 4th Quart., 2020.
- [8] Z. Lv, D. Chen, R. Lou, and Q. Wang, "Intelligent edge computing based on machine learning for smart city," *Future Gener. Comput. Syst.*, vol. 115, pp. 90–99, Feb. 2021.
- [9] C. Wu, Z. Liu, F. Liu, T. Yoshinaga, Y. Ji, and J. Li, "Collaborative learning of communication routes in edge-enabled multi-access vehicular environment," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 4, pp. 1155–1165, Dec. 2020.
- [10] X. Wang, Y. Han, V. C. Leung, D. Niyato, X. Yan, and X. Chen, "Convergence of edge computing and deep learning: a comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 2, pp. 869–904, 2nd Quart., 2020.
- [11] S. Deng, H. Zhao, W. Fang, J. Yin, S. Dustdar, and A. Y. Zomaya, "Edge intelligence: The confluence of edge computing and artificial intelligence," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7457–7469, Aug. 2020.
- [12] C. Gong, F. Lin, X. Gong, and Y. Lu, "Intelligent cooperative edge computing in Internet of Things," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9372–9382, Oct. 2020.
- [13] X. Wang, X. Ren, C. Qiu, Y. Cao, T. Taleb, and V. C. Leung, "Net-in-AI: A computing-power networking framework with adaptability, flexibility, and profitability for ubiquitous AI," *IEEE Netw.*, vol. 35, no. 1, pp. 280–288, Jan./Feb. 2021.
- [14] A. Ignatov, C.-M. Chiang, H.-K. Kuo, A. Sycheva, and R. Timofte, "Learned smartphone ISP on mobile npus with deep learning, mobile ai 2021 challenge: Report," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 2503–2514.
- [15] A. Ignatov, R. Timofte, M. Denna, and A. Younes, "Real-time quantized image super-resolution on mobile NPUS, mobile AI 2021 challenge: Report," *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 2525–2534.
- [16] J. Xu, L. Chen, and P. Zhou, "Joint service caching and task offloading for mobile edge computing in dense networks," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2018, pp. 207–215.
- [17] H. Lin, S. Zeadally, Z. Chen, H. Labiod, and L. Wang, "A survey on computation offloading modeling for edge computing," *J. Netw. Comput. Appl.*, vol. 169, Nov. 2020, Art. no. 102781.
- [18] L. U. Khan, I. Yaqoob, N. H. Tran, S. A. Kazmi, T. N. Dang, and C. S. Hong, "Edge-computing-enabled smart cities: A comprehensive survey," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 10200–10232, Oct. 2020.
- [19] B. Zong, C. Fan, X. Wang, X. Duan, B. Wang, and J. Wang, "6G technologies: Key drivers, core requirements, system architectures, and enabling technologies," *IEEE Veh. Technol. Mag.*, vol. 14, no. 3, pp. 18–27, Sep. 2019.
- [20] J. Wang, K. Zhu, and E. Hossain, "Green Internet of Vehicles (IoV) in the 6G era: Toward sustainable vehicular communications and networking," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 1, pp. 391–423, Mar. 2022.
- [21] Z. Ning *et al.*, "Mobile edge computing enabled 5G health monitoring for Internet of Medical Things: A decentralized game theoretic approach," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 2, pp. 463–478, Feb. 2021.
- [22] D. Wang, N. Zhao, B. Song, P. Lin, and F. R. Yu, "Resource management for secure computation offloading in softwarized cyber-physical system," *IEEE Internet Things J.*, vol. 8, no. 11, pp. 9294–9304, Jun. 2021.
- [23] X. Xiong, K. Zheng, L. Lei, and L. Hou, "Resource allocation based on deep reinforcement learning in IoT edge computing," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 6, pp. 1133–1146, Jun. 2020.
- [24] D. Wang, H. Qin, B. Song, K. Xu, X. Du, and M. Guizani, "Joint resource allocation and power control for D2D communication with deep reinforcement learning in MCC," *Phys. Commun.*, vol. 45, Apr. 2021, Art. no. 101262.
- [25] Z. Ning *et al.*, "Intelligent edge computing in Internet of Vehicles: A joint computation offloading and caching solution," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2212–2225, Apr. 2021.
- [26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [27] X. Hou *et al.*, "Reliable computation offloading for edge-computing-enabled software-defined IoV," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7097–7111, Aug. 2020.
- [28] A. Hammoud, H. Sami, A. Mourad, H. Otrouk, R. Mizouni, and J. Bentahar, "AI, blockchain, and vehicular edge computing for smart and secure IoV: Challenges and directions," *IEEE Internet Things Mag.*, vol. 3, no. 2, pp. 68–73, Jun. 2020.
- [29] N. Kumar, R. Chaudhry, O. Kaiwartya, N. Kumar, and S. H. Ahmed, "Green computing in software defined social Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 6, pp. 3644–3653, Jun. 2021.
- [30] X. Xu, Y. Xue, X. Li, L. Qi, and S. Wan, "A computation offloading method for edge computing with vehicle-to-everything," *IEEE Access*, vol. 7, pp. 131068–131077, 2020.
- [31] A. Muhammad, T. A. Khan, K. Abbass, and W.-C. Song, "An end-to-end intelligent network resource allocation in IoV: A machine learning approach," in *Proc. IEEE 92nd Veh. Technol. Conf. (VTC-Fall)*, 2020, pp. 1–5.
- [32] M. LiWang, Z. Gao, S. Hosseinalipour, H. Dai, and X. Wang, "Energy-aware allocation of graph jobs in vehicular cloud computing-enabled software-defined IoV," in *Proc. IEEE INFOCOM Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2020, pp. 604–609.
- [33] S.-S. Lee and S. Lee, "Resource allocation for vehicular fog computing using reinforcement learning combined with heuristic information," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 10450–10464, Oct. 2020.
- [34] X. Xu, Z. Fang, L. Qi, X. Zhang, Q. He, and X. Zhou, "TripRes: Traffic flow prediction driven resource reservation for multimedia IoV with edge computing," *ACM Trans. Multimedia Comput. Commun. Appl.*, vol. 17, no. 41, pp. 1–21, 2021.
- [35] Z. Lv, D. Chen, and Q. Wang, "Diversified technologies in Internet of Vehicles under intelligent edge computing," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2048–2059, Apr. 2021.
- [36] X. Xu *et al.*, "An edge computing-enabled computation offloading method with privacy preservation for Internet of connected vehicles," *Future Gener. Comput. Syst.*, vol. 96, pp. 89–100, Jul. 2019.
- [37] Q. Wu, X. Xu, Q. Zhao, and F. Dai, "Tasks offloading for connected autonomous vehicles in edge computing," *Mobile Netw. Appl.*, pp. 1–10, Aug. 2021, doi: [10.1007/s11036-021-01794-6](https://doi.org/10.1007/s11036-021-01794-6).
- [38] M. Yang *et al.*, "Deep reinforcement learning based green resource allocation mechanism in edge computing driven power Internet of Things," in *Proc. Int. Wireless Commun. Mobile Comput. (IWCMC)*, 2020, pp. 388–393.
- [39] S. Wan, X. Li, Y. Xue, W. Lin, and X. Xu, "Efficient computation offloading for Internet of Vehicles in edge computing-assisted 5G networks," *J. Supercomput.*, vol. 76, no. 4, pp. 2518–2547, 2020.
- [40] N. Xue, H. Zhang, C. Zhang, T. Li, and D. Yuan, "Data-driven energy efficient predictive resource allocation in Internet of Vehicles," in *Proc. Int. Conf. Wireless Commun. Signal Process. (WCSP)*, 2020, pp. 56–61.
- [41] G. Wang, F. Xu, and C. Zhao, "Multi-access edge computing based vehicular network: Joint task scheduling and resource allocation strategy," in *Proc. IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, 2020, pp. 1–6.
- [42] B. Cao, Z. Sun, J. Zhang, and Y. Gu, "Resource allocation in 5G IoV architecture based on SDN and fog-cloud computing," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 6, pp. 3832–3840, Jun. 2021.
- [43] W. U. Khan, M. A. Javed, T. N. Nguyen, S. Khan, and B. M. Elhalawany, "Energy-efficient resource allocation for 6G backscatter-enabled NOMA IoV networks," *IEEE Trans. Intell. Transp. Syst.*, early access, Sep. 21, 2021, doi: [10.1109/TITS.2021.3110942](https://doi.org/10.1109/TITS.2021.3110942).
- [44] H. Liang *et al.*, "Reinforcement learning enabled dynamic resource allocation in the Internet of Vehicles," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 4957–4967, Jul. 2021.
- [45] K. Wang, X. Wang, X. Liu, and A. Jolfaei, "Task offloading strategy based on reinforcement learning computing in edge computing architecture of Internet of Vehicles," *IEEE Access*, vol. 8, pp. 173779–173789, 2020.
- [46] Q. Qi *et al.*, "Knowledge-driven service offloading decision for vehicular edge computing: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 4192–4203, May 2019.
- [47] K. Wang, X. Wang, and X. Liu, "A high reliable computing offloading strategy using deep reinforcement learning for IoVs in edge computing," *J. Grid Comput.*, vol. 19, no. 2, pp. 1–15, 2021.
- [48] X. Chen, C. Wu, Z. Liu, N. Zhang, and Y. Ji, "Computation offloading in beyond 5G networks: A distributed learning framework and applications," *IEEE Wireless Commun.*, vol. 28, no. 2, pp. 56–62, Apr. 2021.
- [49] A. V. Zelst, R. V. Nee, and G. A. Awater, "Space division multiplexing (SDM) for OFDM systems," in *Proc. IEEE 51st Veh. Technol. Conf.*, vol. 2, 2000, pp. 1070–1074.



- [50] K. Lin, J. Luo, L. Hu, M. S. Hossain, and A. Ghoneim, "Localization based on social big data analysis in the vehicular networks," *IEEE Trans. Ind. Informat.*, vol. 13, no. 4, pp. 1932–1940, Aug. 2017.
- [51] X. Zhang, J. Zhang, Z. Liu, Q. Cui, X. Tao, and S. Wang, "MDP-based task offloading for vehicular edge computing under certain and uncertain transition probabilities," *IEEE Trans. Veh. Technol.*, vol. 69, no. 3, pp. 3296–3309, Mar. 2020.



**Yingjie Liu** received the B.S. degree in electronic and information engineering from Hebei Agricultural University, Baoding, China, in 2019. He is currently pursuing the Ph.D. degree in communication and information systems with Xidian University, Xi'an, China.

His current research interests include causal inference, resource allocation, and reinforcement learning in wireless networks.



**Dan Wang** received the Ph.D. degree in communication and information systems from Xidian University, Xi'an, China, in 2021. From 2019 to 2020, she was a visiting Ph.D. student with the Department of Electrical and Computer Engineering, Carleton University, Ottawa, ON, Canada.

She is currently a Lecturer with the State Key Laboratory of Integrated Services Networks, Xidian University. Her current research interests include mobile-edge computing, resource allocation, and reinforcement learning in wireless networks.



**Bin Song** (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees in communication and information systems from Xidian University, Xi'an, China, in 1996, 1999, and 2002, respectively.

He is currently a Professor with Xidian University. He has authored over 100 journal papers or conference papers and 50 patents. His research interests are in multimedia communication, multimodal data fusion, content-based image recognition and machine learning, reinforcement learning, Internet of Things, big data, and recommendation systems.



**Xiaojiang (James) Du** (Fellow, IEEE) received the B.S. degree from Tsinghua University, Beijing, China, in 1996, and the M.S. and Ph.D. degrees in electrical engineering from the University of Maryland at College Park in 2002 and 2003, respectively.

He is the Anson Wood Burchard Endowed-Chair Professor with the Department of Electrical and Computer Engineering, Stevens Institute of Technology. He was a Tenured Professor with Temple University from August 2009 to August

2021. His research interests are security, wireless networks, and systems. He has authored over 500 journal and conference papers in these areas, including the top security conferences IEEE S&P, USENIX Security, and NDSS. He has been awarded more than U.S. \$8 million research grants from the U.S. National Science Foundation, Army Research Office, Air Force Research Lab, the State of Pennsylvania, and Amazon. He won the Best Paper Award at several conferences, such as IEEE ICC 2020 and IEEE GLOBECOM 2014 and the Best Poster Runner-Up Award at ACM MobiHoc 2014. He serves on the editorial boards of three IEEE journals. He is an ACM Distinguished Member and an ACM Life Member.